Experiment 'p_6_7_reverse_array_array' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_7_reverse_array_array
Natural language statement: Write a method that reverses the sequence of elements in an array. For example, if you call the method with the array [1, 4, 9, 16, 9, 7, 4, 9, 11] then the array is changed to [11, 9, 4, 7, 9, 16, 9, 4, 1].
Method signature: p_6_7_reverse_array_array(inputs: array<int>) returns (reversed: array<int>)

Ensures

- `reversed.Length == inputs.Length`
- `forall i :: 0 <= i < reversed.Length ==> reversed[i] == inputs[inputs.Length - 1 - i]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that reverses the sequence of elements in an array. For example, if you
    ↪ call the method with the array [1, 4, 9, 16, 9, 7, 4, 9, 11] then the array is
    ↪ changed to [11, 9, 4, 7, 9, 16, 9, 4, 1].

The signature should be:

p_6_7_reverse_array_array(inputs: array<int>) returns (reversed: array<int>)

The method should respect the following contract:

ensures reversed.Length == inputs.Length, ensures forall i :: 0 <= i < reversed.Length
    ↪ ==> reversed[i] == inputs[inputs.Length - 1 - i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn1hMrYlbsoMdkiL9pOtncF2gKBXq
Model: gpt-5.5-2026-04-23
Created at: 1780576972
Finish reason: stop
Usage: 458 tokens in, and 484 tokens out

```
{
  reversed := new int[inputs.Length];
  var i := 0;
  while i < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant reversed.Length == inputs.Length
    invariant forall j :: 0 <= j < i ==> reversed[j] == inputs[inputs.Length - 1 - j]
  {
    reversed[i] := inputs[inputs.Length - 1 - i];
    i := i + 1;
  }
}
```

Final Program

```
method p_6_7_reverse_array_array(inputs: array<int>) returns (reversed: array<int>)
```

```
      ensures reversed.Length == inputs.Length
      ensures forall i :: 0 <= i < reversed.Length ==> reversed[i] == inputs[inputs.
          ↪ Length - 1 - i]
{
  reversed := new int[inputs.Length];
  var i := 0;
  while i < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant reversed.Length == inputs.Length
    invariant forall j :: 0 <= j < i ==> reversed[j] == inputs[inputs.Length - 1 - j]
  {
    reversed[i] := inputs[inputs.Length - 1 - i];
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 458
Output tokens: 484
Reasoning tokens: 361
Sum of 'total tokens': 942

Experiment Timings

Overall Experiment started at 1780576971406, ended at 1780577010819, lasting 39413ms (39.41 seconds)
Iteration #1 started at 1780576971406, ended at 1780577010819, lasting 39413ms (39.41 seconds)